

Lezione 17 - Python e funzioni

GENERATORI

◆ Idea

👉 Quando iteriamo:

- spesso NON serve avere tutta la lista/array in memoria
- soprattutto se è molto grande o infinita

Meglio:

✳ generare gli oggetti/valori uno alla volta

◆ Problema

👉 Una lista:

- viene creata tutta subito
- occupa memoria

Esempio:

```
[x for x in range(1000000)]
```

✳ crea tutti gli elementi in memoria

◆ Soluzione: GENERATORI

👉 I generatori:

- producono valori uno alla volta
- NON salvano tutto

✳️ più efficienti

📌 COROUTINE

👉 Un generatore è un caso particolare di:

✳️ **coroutine**

◆ Proprietà delle coroutine

👉 Una coroutine ha due *principali caratteristiche*:

- Mantiene lo stato delle variabili locali
- Può essere sospesa quando il controllo esce al di fuori della coroutine
- Può riprendere l'esecuzione da dove si era interrotta

✳️ NON ricomincia da capo

📌 YIELD

👉 La parola chiave nel caso dei **generatori** è:

`yield`

👉 Serve per:

- Restituire un valore

- Sospendere la funzione

👉 Infatti alla chiamata successiva:

- La funzione riprende da lì

✳️ esecuzione “a pezzi”

ESEMPIO BASE

Codice:

```
def simple_gen():  
    yield 1  
    yield 2  
    yield 3
```

```
x = simple_gen()
```

```
print(next(x))  
print(next(x))  
print(next(x))
```

◆ Osservazione

👉 Il generatore:

- Restituisce un valore alla volta
- Si ferma su `yield`
- Riprende alla chiamata successiva

👉 L'esempio precedente è semplice. Ora vediamo un caso più importante in cui il generatore mantiene uno stato interno

📌 ESEMPIO PRINCIPALE (Dove si vede lo stato)



Codice:

```
def f():  
    k = 1  
    while True:  
        yield k  
        k += 1
```

```
x = f()
```

```
print(next(x))  
print(next(x))
```

◆ Cosa succede

👉 Alla prima chiamata:

- k = 1
- restituisce 1

👉 Alla seconda chiamata:

- NON riparte da capo
- continua da dove era

👉 quindi:

- k = 2
- restituisce 2

✳ Il valore viene aggiornato e mantenuto

IDEA IMPORTANTE

👉 I generatori:

- Interrompono e riprendono l'esecuzione
- Mantengono lo stato interno

Ora vediamo meglio cosa significa che una funzione mantiene lo stato e come funziona l'esecuzione passo dopo passo con `next()`

NEXT E STOPITERATION

◆ `next()`

👉 Per ottenere i valori da un generatore usiamo la funzione:

`next(x)`

👉 Ogni chiamata:

- Esegue la funzione fino al prossimo `yield`
- Restituisce il valore

◆ Fine del generatore

👉 Quando non ci sono più `yield` :

✳ Viene sollevata un'eccezione

👉 Tipo:

StopIteration



Esempio:

```
def simple_gen():  
    yield 1  
    yield 2  
  
x = simple_gen()  
  
print(next(x)) # 1  
print(next(x)) # 2  
print(next(x)) # errore
```

◆ Comportamento

👉 Il generatore:

- Mantiene lo stato
- Termina quando finisce il codice

✳ Dopo la fine → non produce più valori



GENERATOR COMPREHENSION

👉 Come le list comprehension:

```
[x for x in range(10)]
```

👉 esiste anche:

```
(x for x in range(10))
```

◆ Differenza

Tipo	Comportamento
Lista	crea tutto subito
Generatore	produce valori uno alla volta

✳ Il generatore è più efficiente

📌 IDEA IMPORTANTE

👉 Generatori:

- Lavorano “on demand”
- Producono valori solo quando servono

Ora generalizziamo questo meccanismo: vediamo come funzionano gli **iteratori** in Python

📌 ITERATORI

◆ Idea

👉 Non solo le funzioni con `yield` possono generare valori

👉 Anche gli oggetti possono comportarsi come generatori

✳️ usando il **protocollo degli iteratori**

◆ ITERATOR PROTOCOL

👉 Un oggetto è un iteratore se implementa due **metodi**:

◆ `__iter__()` **{#-iter }**

👉 serve per:

- Inizializzare l'iterazione
- Restituire l'oggetto iteratore

◆ `__next__()` **{#-next }**

👉 serve per:

- Restituire il valore successivo
- oppure sollevare `StopIteration`

📌 IDEA GUIDA

👉 Vogliamo creare un oggetto che:

- contiene dati
- possiamo scorrerlo con un `for`

✳️ come una lista

◆ Struttura della Classe



Codice:

```
class MySet:  
  
    def __init__(self):  
        self._data = []
```

👉 `_data` :

- contiene gli elementi

◆ Aggiunta elementi



Codice:

```
def add(self, x):  
    if x not in self._data:  
        self._data.append(x)
```

👉 simula un insieme:

- niente duplicati

◆ ITERAZIONE (PARTE 1)



Codice:

```
def __iter__(self):  
    self._pos = 0  
    return self
```

👉 cosa fa:

- inizializza la posizione (`_pos`)
- restituisce l'iteratore

◆ ITERAZIONE (PARTE 2)



Codice:

```
def __next__(self):  
    if self._pos >= len(self._data):  
        raise StopIteration
```

👉 se abbiamo finito:

✳ stop



Codice:

```
res = self._data[self._pos]  
self._pos += 1  
return res
```

👉 altrimenti:

- restituisce il valore corrente
- passa al successivo

UTILIZZO

Codice:

```
s = MySet()  
s.add(100)  
s.add(89)  
s.add(42)
```

```
for x in s:  
    print(x)
```

👉 il for :

- chiama `__iter__()`
- poi chiama `__next__()` finché non trova `StopIteration`

✳️ stesso meccanismo dei generatori

UTILIZZO MANUALE

Codice:

```
x = iter(s)  
print(next(x))
```

👉 equivalente al for

◆ COSA NOTARE

👉 Questo oggetto:

- Si comporta come un generatore
- Mantiene uno stato (`_pos`)

📌 COLLEGAMENTO

👉 I generatori:

- Implementano automaticamente questo protocollo

👉 tramite:

`yield`

mentre

👉 Negli Iteratori

- Bisogna implementare il protocollo manualmente

👉 tramite:

(`__next__`)

📌 IDEA IMPORTANTE

👉 iteratori e generatori:

- fanno la stessa cosa
- ma in modo diverso

✳️ stesso concetto, implementazione diversa

Ora vediamo strumenti pronti per lavorare con iteratori: la libreria **itertools**

ITERTOOLS

◆ Idea

`itertools` è una libreria che:

- Fornisce iteratori già pronti
- Permette di lavorare su sequenze in modo efficiente

◆ Import

 Codice:

```
from itertools
import product,
    chain,
    pairwise,
    permutations,
    combinations,
    repeat
```

PRINCIPALI FUNZIONI

◆ `chain(a, b, ...)`

- Itera su tutti gli elementi di `a`, poi `b`, ecc.
- Equivalente a concatenare sequenze



Codice:

```
for i in chain(a, b, c):  
    print(i)
```

◆ **product(a, b, ...)**

- Itera su tutte le combinazioni di valori a , b , ecc.
- Equivalente a cicli annidati



Codice:

```
for i, j in product(a, b):  
    print(i, j)
```

◆ **pairwise(a)**

- Produce coppie consecutive: (a[0], a[1]), (a[1], a[2]), ...



Codice:

```
for i, j in pairwise(a):  
    print(i, j)
```

◆ **permutations(a)**

- Genera tutte le permutazioni degli elementi
- Ordine importante
- Numero di risultati: $n!$



Codice:

```
for i in permutations(a):  
    print(i)
```

◆ combinations(a, m)

- Itera su tutte le tuple ordinate di m elementi di a
- senza ripetizioni
- L'ordine NON conta



Codice:

```
for i in combinations(a, 2):  
    print(i)
```

◆ repeat(x, m)

- Ripete il valore x per m volte
- Se m non è specificato l'iteratore è infinito



Codice:

```
for i in repeat("ciao", 10):  
    print(i)
```



IDEA IMPORTANTE

◆ Tutte queste funzioni:

- Restituiscono iteratori
- NON creano liste complete

◆ Vantaggio

- Meno memoria
- Esecuzione “on demand”

Ora passiamo a un concetto fondamentale: le **funzioni come oggetti**

FUNZIONI COME FIRST-CLASS OBJECTS

◆ Idea

In Python le funzioni sono oggetti.

Questo significa che possono:

- Essere assegnate a una variabile
- Essere restituite da un'altra funzione
- Essere passate come argomento

FUNZIONI INNESTATE

◆ Idea

Possiamo definire una funzione dentro un'altra funzione.

La funzione interna:

- Esiste localmente
- Può essere restituita
- Può usare variabili della funzione esterna

◆ Funzione che restituisce una funzione



Codice:

```
def f():  
    def g(x):  
        return x + 4  
    return g
```

👉 f definisce la funzione g

👉 f restituisce la funzione g

```
h = f()
```

👉 h diventa la funzione g

```
print(h(3))
```

👉 Esegue g(3)

👉 Risultato: 5



CLOSURES

◆ Idea

Una **closure** è una funzione che “si ricorda” l’ambiente in cui è stata creata.

In pratica:

- Abbiamo una funzione interna
- Questa usa una variabile della funzione esterna
- Il valore viene mantenuto anche dopo la fine della funzione esterna

◆ Significato

La closure è composta da due parti:

- Una funzione f
- Un ambiente che a ogniuna delle variabili libere di f associa il valore che aveva al momento della definizione di f

◆ Esempio



Codice:

```
def f(x):  
    def g(y):  
        return x + y  
    return g
```

👉 g usa la variabile x

```
h = f(5)
```

👉 Crea una funzione con $x = 5$

```
print(h(3))
```

👉 Esegue $5 + 3$

👉 Risultato: 8

ESEMPIO CON LISTE

Codice:

```
from copy import copy
```

👉 Importiamo `copy` per duplicare liste

```
def list_concat(first):  
    def concat(second):  
        res = copy(first)  
        res.append(second)  
        return res  
    return concat
```

👉 `concat` cattura `first`

👉 Crea una copia e aggiunge elementi

```
x = [1, 2, 3]  
c = list_concat(x)
```

👉 `c` diventa una funzione

👉 “ricorda” la lista `x`

```
print(c([5, 6, 7]))
```

👉 Copia `[1,2,3]`

👉 Aggiunge `[5,6,7]`

```
x[0] = 12  
print(c([5, 6, 7]))
```

👉 Modifica `x`

👉 La funzione usa il nuovo valore

FUNZIONI ANONIME

◆ Idea

In Python possiamo definire funzioni senza dare loro un nome.

Queste funzioni si chiamano:

- **funzioni anonime**
- oppure **lambda functions**

◆ Sintassi

Codice:

```
lambda x: x + 3
```

◆ Significato

Nella lambda:

- `x` è l'argomento
- `x + 3` è il corpo della funzione

Il `return` è implicito.

◆ Limitazioni

Le lambda in Python:

- Sono limitate a una sola espressione
- Non permettono codice complesso
- Non permettono più istruzioni

◆ Idea importante

Le funzioni anonime sono utili quando:

- La funzione è molto corta
- Serve solo in un punto
- Vogliamo passarla rapidamente ad altre funzioni



Esempio:

```
f = lambda x: x + 2  
print(f(3))
```

👉 $f(3) \rightarrow 5$



Esempio:

```
g = lambda x, y: x + 2*y  
print(g(3, 4))
```

👉 $g(3,4) \rightarrow 11$

Ora vediamo come usare le funzioni su collezioni: **map**, **filter**, **reduce** e **partial**

LAVORARE CON FUNZIONI

◆ Idea

Molte operazioni su collezioni sono riconducibili a particolari applicazioni di funzioni su di esse:

◆ MAP(f, collection)

👉 Applica la funzione `f` a tutti gli elementi della collezione

Esempio:

```
map_res = map(lambda x: x + 3, lst)
print(list(map_res))
```

👉 applica `x + 3` a ogni elemento di `lst`

👉 `map` restituisce un iteratore → convertiamo con `list()`

◆ FILTER (predicate, collection)

👉 Applica il predicato (funzione che ritorna valori Booleani) agli elementi della collezione e mantiene solo quelli per cui il predicato è vero

Esempio:

```
filter_res = filter(lambda x: x % 2 == 0, lst)
print(list(filter_res))
```

👉 Tiene solo i numeri pari

👉 Scarta gli altri

REDUCE

◆ REDUCE (f, collection)

👉 Usa la funzione `f` (che ha due argomenti) per ridurre la collezione ad un solo valore:

- Per usarlo in python serve importarlo

Codice per importare la REDUCE:

```
from functools import reduce
```

Esempio:

```
reduce_res = reduce(lambda x, y: x + y, lst)
print(reduce_res)
```

👉 somma tutti gli elementi

👉 esegue: $((1+2)+3)+\dots$

◆ PARTIAL

👉 Fissa alcuni parametri di una funzione

- Per usarlo in Python serve importarlo

Codice per importare PARTIAL:

```
from functools import partial
```



Esempio:

```
def somma(x, y):  
    return x + y
```

```
somma3 = partial(somma, 3)  
print(somma3(5))
```

👉 equivale a: `somma(3, 5)`

👉 risultato: 8



IDEA IMPORTANTE

◆ Tutte queste funzioni

👉 Lavorano con funzioni

👉 Spesso usano lambda

Ora vediamo come modificare il comportamento delle funzioni: i **decoratori**



DECORATORI

◆ Idea

👉 Un decoratore è una funzione che:

- Prende una funzione
- Restituisce una nuova funzione

👉 Serve per:

- Modificare il comportamento

- Aggiungere funzionalità

👉 Permettono di:

- Modificare funzioni
- Senza cambiare il codice originale

👉 Sono basati su:

- Funzioni come oggetti
- Closures

DECORATORE BASE

Codice:

```
def decorator(f):  
    def wrapped(*args, **kwargs):  
        return f(*args, **kwargs)  
    return wrapped
```

- 👉 decorator prende una funzione f
- 👉 restituisce una nuova funzione wrapped

UTILIZZO

Codice:

```
@decorator  
def f(x, y):  
    return x + y
```

- 👉 equivale a:

```
f = decorator(f)
```



Codice:

```
def wrap_function(f):  
    def wrapped(*args, **kwargs):  
        print("Sto per chiamare la funzione")  
        x = f(*args, **kwargs)  
        print("Ho chiamato la funzione")  
        return x  
    return wrapped
```

👉 Aggiunge messaggi prima e dopo la funzione

```
@wrap_function  
def adder(x, y):  
    return x + y
```

```
adder(2, 3)
```

👉 Stampa messaggi + risultato

DECORATORE CON LOG

Codice:

```
def logger(f):  
    def add_logger(*args, **kwargs):  
        with open("log.txt", "a") as log:  
            log.write(f"calling with args {args} and {kwargs}\n")  
            res = f(*args, **kwargs)  
            log.write(f"result: {res}\n")  
            return res  
    return add_logger
```

 Salva informazioni su file

```
@logger  
def list_product(lst):  
    p = 1  
    for x in lst:  
        p *= x  
    return p
```

 Ogni chiamata viene loggata

DECORATORI CON PARAMETRI

Codice:

```
from functools import wraps
```

 Per importare

```

def better_logger(filename):
    def logger(f):
        @wraps(f)
        def add_logger(*args, **kwargs):
            with open(filename, "a") as log:
                log.write(f"calling with args {args} and {kwargs}\n")
                res = f(*args, **kwargs)
                log.write(f"result: {res}\n")
            return res
        return add_logger
    return logger

```

👉 Decoratore che riceve parametri

```

@better_logger("mylog.txt")
def list_product2(lst):
    p = 1
    for x in lst:
        p *= 2*x
    return p

```

👉 Crea un decoratore personalizzato

CACHE (ESEMPIO AVANZATO)

 **Codice:**

```

caches = {}

```

```
def cached(f):
    name = f.__name__
    if name not in caches:
        caches[name] = {}

    def with_cache(x):
        if x in caches[name]:
            return caches[name][x]

        res = f(x)
        caches[name][x] = res
        return res

    return with_cache
```

👉 Salva risultati già calcolati

```
@cached
def f(x):
    for i in range(100_000_000):
        x += i
    return x
```

👉 Evita ricalcoli